

```
name: maintainer-absence
```

```
on:
```

```
  workflow_dispatch:
```

```
    inputs:
```

```
      startDate:
```

```
        description: 'First day of maintainer absence [mm-dd-yyyy]'
```

```
        required: true
```

```
      endDate:
```

```
        description: 'Last day of maintainer absence [mm-dd-yyyy]'
```

```
        required: true
```

```
permissions:
```

```
  issues: write
```

```
jobs:
```

```
  create-issue:
```

```
    name: create-issue
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/github-script@v9
```

```
        with:
```

```
          script: |
```

```
            const startDate = new Date('${{
github.event.inputs.startDate }}');
```

```
            const endDate = new Date('${{
github.event.inputs.endDate }}');
```

```
            if (startDate > endDate) {
```

```
              throw 'Start date cannot be later than end date.';
```

```
            }
```

```
            // Calculate total days of absence
```

```
            const differenceInDays = endDate.getTime() -
startDate.getTime();
```

```
            const lengthOfAbsence = differenceInDays/(1000 *
3600 * 24);
```

```
// Create issue
issue = await github.rest.issues.create({
  owner: context.repo.owner,
  repo: context.repo.repo,
  // Use the briefer input date format in title
  // (instead of JavaScript's full date string)
  title: Maintainer(s) will be away from ${
    github.event.inputs.startDate } until ${
    github.event.inputs.endDate },
  body: The ${context.repo.repo} maintainer(s)
    will be away for ${lengthOfAbsence}
    day${lengthOfAbsence > 1 ? 's' : ''} beginning on
    ${startDate.toDateString()} and ending on
    ${endDate.toDateString()}. During this time, the
    maintainer(s)
    will not be actively monitoring PRs, discussions,
    etc. Please report any issues
    requiring immediate attention to
    [@GitCredManager](https://twitter.com/GitCredManager)
    on Twitter.
});
```

```
// Pin issue - we use GraphQL since there is no
GitHub API available for this
```

```
const mutation = mutation($issueId: ID!) {
  pinIssue(input: { issueId: $issueId }) {
    issue {
      repository {
        id
      }
    }
  }
};
const variables = {
  issueId: issue.data.node_id
}
const result = await github.graphql(mutation,
variables)
```

Show less ^

This GitHub Actions workflow creates and pins a GitHub issue announcing that the project maintainer(s) will be unavailable for a specified period. Here's a detailed explanation.

Workflow Overview

Workflow Name

</> YAML



```
name: maintainer-absence
```

This is the name that appears in the GitHub Actions tab.

Trigger

</> YAML



```
on:
  workflow_dispatch:
```

The workflow is **manually triggered** from GitHub.

When clicking **Actions** → **maintainer-absence** → **Run workflow**, GitHub asks for two inputs.

</> YAML



```
inputs:
  startDate:
    description: 'First day of maintainer absence [mm-dd-yyyy]'
    required: true

  endDate:
    description: 'Last day of maintainer absence [mm-dd-yyyy]'
    required: true
```

Example:

```
Start Date : 07-15-2026
End Date   : 07-25-2026
```



Permissions

</> YAML



```
permissions:
  issues: write
```

The workflow is allowed to

- Create issues
- Edit issues
- Pin issues (via GraphQL)

Without this permission, the issue creation would fail.

Job

</> YAML



```
jobs:
  create-issue:
```

There is only one job.

Runner

</> YAML



```
runs-on: ubuntu-latest
```

GitHub creates a temporary Ubuntu virtual machine.

Step

</> YAML



```
- uses: actions/github-script@v9
```

Instead of writing JavaScript in a separate file, this action lets you execute JavaScript directly inside the workflow.

It automatically provides:

- `github`
 - `context`
 - authentication using `GITHUB_TOKEN`
-

Reading Inputs

</> JavaScript



```
const startDate = new Date('${{ github.event.inputs.startDate }}');  
const endDate = new Date('${{ github.event.inputs.endDate }}');
```

Suppose the user entered

07-15-2026

07-25-2026



Then

</> JavaScript



```
startDate = Tue Jul 15 2026  
endDate   = Sat Jul 25 2026
```

Date Validation

</> JavaScript



```
if (startDate > endDate) {  
    throw 'Start date cannot be later than end date.';  
}
```

If the user enters

Start : 07-30-2026

End : 07-20-2026



the workflow immediately fails.

Calculate Duration

JavaScript



```
const differenceInDays =  
    endDate.getTime() - startDate.getTime();
```

`getTime()` returns milliseconds since January 1, 1970.

Example:

July 25 - July 15



= 10 days

Convert milliseconds into days:

JavaScript



```
const lengthOfAbsence =  
    differenceInDays / (1000 * 3600 * 24);
```

where:

- 1000 = milliseconds per second
- 3600 = seconds per hour
- 24 = hours per day

Note

This calculation is **exclusive** of the start date.

Example:

15 → 25



returns 10

If you want to count **both** July 15 and July 25, use:

JavaScript



```
const lengthOfAbsence =  
    differenceInDays / (1000 * 3600 * 24) + 1;
```

Create GitHub Issue

</> JavaScript



```
issue = await github.rest.issues.create({
```

Uses the GitHub REST API.

Repository:

</> JavaScript



```
owner: context.repo.owner,  
repo: context.repo.repo,
```

For example

```
owner = microsoft
```



```
repo = vscode
```

Issue Title

</> JavaScript



```
title:  
`Maintainer(s) will be away from  
07-15-2026 until 07-25-2026`
```

Issue Body

</> JavaScript



```
body:  
`The repository maintainer(s)  
will be away for 10 days...`
```

Produces something like:

```
The vscode maintainer(s) will be away for 10 days  
beginning on Wed Jul 15 2026  
and ending on Sat Jul 25 2026.
```



```
During this time the maintainer(s)  
will not be actively monitoring PRs,  
issues or discussions.
```

```
Please report urgent issues to  
@GitCredManager.
```

Returned Response

```
</> JavaScript
```



```
issue.data
```

contains the created issue.

Example:

```
</> JSON
```



```
{  
  "number": 123,  
  "id": 9999999,  
  "node_id": "MDU6SXNzdWU5OTk5OTk5"  
}
```

The important value is

```
</> JavaScript
```



```
issue.data.node_id
```

which is required by GraphQL.

Pinning the Issue

GitHub's REST API does **not** support pinning issues, so the workflow uses the GraphQL API.

GraphQL Mutation

</> JavaScript



```
const mutation = `
mutation($issueId: ID!) {
  pinIssue(input: {
    issueId: $issueId
  }) {
    issue {
      repository {
        id
      }
    }
  }
}
`;
```

This defines a GraphQL mutation named `pinIssue` that pins the specified issue.

Variables

</> JavaScript



```
const variables = {
  issueId: issue.data.node_id
}
```

Example:

</> JavaScript



```
{
  issueId:
    "MDU6SXNzdWU50Tk50Tk5"
}
```

Execute Mutation

</> JavaScript



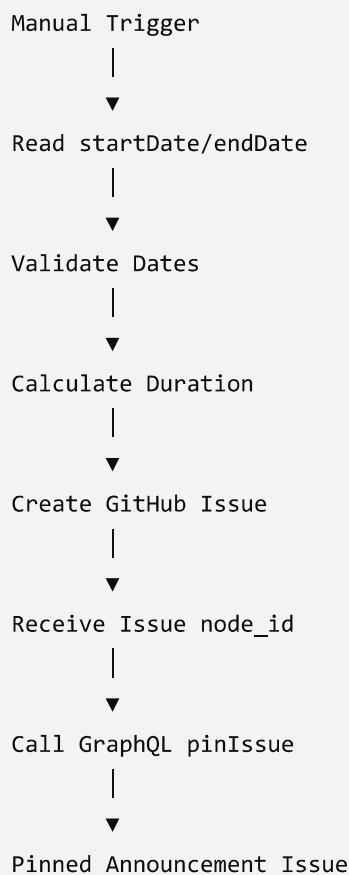
```
const result =
await github.graphql(
```

```
mutation,  
variables  
);
```

This sends the mutation to GitHub.

If successful, the issue becomes pinned at the top of the repository's Issues page.

Execution Flow



Improvements you could make

- **Count days inclusively** by adding `+1` to the day calculation if both the start and end dates should be included.
- **Use ISO date format** (`yyyy-mm-dd`) for inputs instead of `mm-dd-yyyy` , as JavaScript's `Date` parsing can be locale-dependent.
- **Declare variables properly** using `const` or `let` :

</> JavaScript

```
const issue = await github.rest.issues.create({ ... });
```

instead of assigning to an undeclared `issue` .

- **Update the contact link**, as the example uses a Twitter/X profile (`@GitCredManager`) that may no longer be appropriate; consider linking to a GitHub team, organization contact, or discussion instead.
- **Add labels or assign the issue** automatically if you want it categorized (e.g., `announcement` , `maintenance`).

